PSG COLLEGE OF TECHNOLOGY

23Z611 – INNOVATION PRACTICES LABORATORY

An Intelligent Voice-Driven Meeting Assistant for Microsoft teams with Retrieval-Augmented Generation

ASHWATH PRABHU 23Z309
HARIHARAN SRIKANTH CHETTY 23Z323
RHITHANYA P 23Z358
RITHIK R K 23Z359
S SHASHWATH 23Z362

1. Introduction

The modern enterprise relies heavily on digital communication platforms for internal collaboration, employee training, and knowledge dissemination. Microsoft Teams, adopted by over 320 million monthly active users globally, has become the default medium for meetings, onboarding sessions, and cross-functional discussions in organizations of all sizes. However, these virtual interactions remain fundamentally limited by the availability and bandwidth of human participants.

When subject-matter experts are required to repeatedly address recurring queries — particularly during new-hire onboarding, policy clarification sessions, and internal training calls — a significant portion of their productive time is consumed by repetitive, low-complexity tasks. The rapid maturation of large language models, retrieval-augmented generation pipelines, and real-time speech processing technologies has created a viable pathway for intelligent conversational agents capable of participating in multi-party voice calls.

This project addresses the intersection of these two realities:

- The operational burden of repetitive knowledge transfer in enterprises
- The technological readiness of AI-driven conversational systems

The proposed system is an AI-powered meeting assistant that:

- Autonomously joins Microsoft Teams calls
- Interacts with multiple participants through voice
- Answers questions grounded in uploaded company documents using retrieval-augmented generation
- Maintains contextual memory throughout a session
- Logs the entire conversation as a structured transcript

The motivation arises from a concrete organizational need: companies spend disproportionate effort onboarding new employees, who inevitably have similar sets of questions regarding policies, tools, processes, and operational procedures. By deploying an AI agent that can attend these sessions, answer document-backed questions in real time, and maintain a record of all interactions, organizations can significantly reduce the time burden on senior staff while ensuring consistent, accurate, and auditable responses.

2. Company Overview – Redington Limited

Redington Limited is a leading technology distributor and supply chain solutions provider operating across emerging markets. The company manages end-to-end distribution for technology products including IT hardware, mobility devices, consumer electronics, cloud solutions, and software services.

Typical onboarding workflow:

- Scheduled Microsoft Teams calls where 2–5 new hires interact with a senior employee or HR representative
- New hires ask questions about leave policies, compensation structures, tool access procedures, VPN configuration, and organizational norms
- The senior employee answers from memory or by referencing internal documents
- This cycle repeats with every new batch of joiners

Key challenges in this domain:

- Knowledge is already documented but scattered across multiple files, portals, and wikis — making real-time retrieval during a live call impractical for human facilitators
- Quality and consistency of answers vary depending on which employee conducts the session, leading to information asymmetry among new hires
- No systematic logging of what questions were asked and how they were answered, resulting in a loss of institutional memory
- The process does not scale — as the organization grows, onboarding frequency and batch sizes increase, but the pool of available senior employees does not grow proportionally

Beyond onboarding, the system has applicability in internal knowledge-sharing sessions, policy update briefings, and technical training calls, where an AI assistant can provide real-time, document-grounded answers while human facilitators focus on high-level discussion and decision-making.

3. Problem Statement

The core problem addressed by this project is the absence of an intelligent, voice-capable agent that can autonomously participate in Microsoft Teams meetings, interact naturally with multiple participants, and provide accurate answers grounded in organizational documents.

Limitations of current approaches:

- Meeting transcription services produce text records only after the fact — they cannot respond to questions in real time

- Text-based chatbots operate through typed interfaces and do not integrate into the audio flow of a live meeting
- Manual facilitation is effective but unsustainable at scale — a senior employee must simultaneously listen to multiple participants, recall or search for correct information, formulate clear responses, and manage conversational dynamics

Additional complexity in multi-speaker environments:

- When two or three participants speak simultaneously or in rapid succession, a human facilitator must manage turn-taking, prioritize questions, and maintain conversational coherence
- An AI agent intended for this setting must replicate these social-communicative behaviours to function effectively

The project therefore seeks to develop a system that can:

- Join Teams meetings programmatically
- Transcribe participant speech in real time
- Retrieve relevant information from uploaded documents
- Generate contextually appropriate responses
- Synthesize those responses as audible speech
- Manage multi-speaker interactions gracefully
- Log the entire session as a structured, searchable transcript

4. Objectives

- Voice-enabled conversational interaction — Develop an agent that listens to spoken questions via speech-to-text and responds via text-to-speech, creating a seamless voice loop indistinguishable from a human participant in terms of conversational mechanics
- Retrieval-augmented generation — Implement a RAG pipeline that grounds answers in uploaded organizational documents rather than relying solely on the language model's general knowledge. This involves document ingestion, text extraction, semantic chunking, vector embedding, storage, and similarity-based retrieval
- Multi-speaker interaction management — Detect overlapping speech, queue incoming questions, acknowledge each speaker, and process queries sequentially — mimicking the turn-taking behaviour of a human facilitator
- Per-session conversational memory — Maintain isolated conversation history for each meeting, enabling follow-up questions and contextual references without confusing context across sessions

- Comprehensive session logging — Log the complete meeting interaction as a structured group-chat-style transcript with speaker identities, timestamps, utterance content, and response metadata
- Web-based dashboard — Provide a graphical interface for document upload, source management, meeting controls, transcript viewing, and a text-based fallback for testing

5. System Overview

The proposed system functions as an intelligent meeting assistant that operates as a participant within Microsoft Teams calls. It is not a replacement for human trainers, managers, or facilitators; rather, it augments their capacity by handling routine, document-answerable queries autonomously.

What the system does:

- Accepts organizational documents (PDF, DOCX, TXT, HTML) and processes them into a searchable knowledge base
- Joins Teams calls and listens to all participants
- Identifies spoken questions, searches the knowledge base, constructs contextual responses using a large language model, and delivers responses as synthesized speech
- Maintains a running transcript of the entire meeting, structured as a group chat log with speaker attribution

What the system does not do:

- Replace the judgment and authority of human managers and trainers
- Make decisions on behalf of the organization
- Access confidential or restricted data beyond what is explicitly uploaded
- Dual-interface design:
- Voice interface within Microsoft Teams — the agent participates as an audio-capable bot
- Web dashboard built with React — provides administrative capabilities including document management, conversation management, meeting controls, and transcript review

6. System Architecture

The system follows a modular pipeline architecture with data flowing sequentially from audio input through processing stages to audio output, with a parallel data path for web-based interactions.

Pipeline flow:

- Participant audio is captured from the Teams meeting through Azure Communication Services
- Audio is routed to the Speech-to-Text module (Vosk), which transcribes speech into text
- Transcribed text and speaker identity are passed to the Call Manager
- The Call Manager detects overlapping speakers, manages a speech queue, and dequeues questions sequentially
- Each question is forwarded to the RAG Agent, which queries the Vector Database for semantically similar document chunks
- Retrieved chunks + conversation history + the question are assembled into a prompt and submitted to the Large Language Model (Gemini 2.5 Flash)
- The generated response is passed to the Text-to-Speech module (gTTS), which synthesizes audio
- Synthesized audio is injected back into the Teams call through Azure Communication Services
- Throughout the pipeline, every interaction is logged into the transcript

Web dashboard path:

- React frontend communicates with the backend server through REST APIs
- Enables document upload, source management, conversation management, meeting controls, and transcript retrieval
- Provides a text-based chat that bypasses speech processing for testing and fallback

7. Detailed Module Descriptions

7.1 Teams Audio and Media Pipeline

- **Purpose:** Establish and manage the connection between the AI agent and Microsoft Teams meetings
- **Description:** Registers a bot with Azure Active Directory, uses Azure Communication Services Call Automation SDK to join meetings programmatically, and manages bidirectional audio streams
- **Key Functions:**
    - Join a meeting using its Teams URL
    - Leave an active meeting using its connection identifier
    - Receive and process real-time call events via webhook (connection, disconnection, participant updates, speech recognition results)

    - Extract transcribed text and speaker identity from speech events
    - Inject synthesized audio into the active call for playback
- **Inputs:** Teams meeting URLs, Azure event payloads
- **Outputs:** Structured event records with call state, speaker identities, and transcribed speech text

7.2 Speech Processing Service

- **Purpose:** Bidirectional conversion between spoken audio and text
- **Description:** Uses Vosk for offline speech-to-text (no API key, no network latency) and gTTS for text-to-speech (free, natural-sounding)
- **Key Functions:**
    - Transcribe raw audio bytes by processing in 4KB chunks through a streaming recognizer
    - Transcribe WAV files directly
    - Synthesize text into audio as an in-memory byte stream (MP3 format)
    - Report readiness status of both STT and TTS engines
- **Inputs:** Raw audio bytes (PCM 16-bit mono) for STT; plain text strings for TTS
- **Outputs:** Transcribed text strings; synthesized audio bytes

7.3 RAG Agent and Language Model Orchestration

- **Purpose:** Process questions, retrieve relevant document context, and generate grounded responses
- **Description:** Queries the vector database for the five most similar document chunks, constructs a context-augmented prompt with conversation history, and submits it to Gemini 2.5 Flash via LangChain
- **Key Functions:**
    - Retrieve relevant document chunks filtered by conversation and active sources
    - Construct structured prompts with document context, conversation history, and instructions
    - Invoke the language model with retry logic (3 attempts, exponential backoff)
    - Fall back to direct language model call if the retrieval-augmented chain fails
    - Maintain per-session conversation history (last 10 exchanges)
    - Clearly indicate response mode (from documents vs. general knowledge)

- **Inputs:** Question string, optional conversation identifier, optional list of active source names
- **Outputs:** Answer text, list of source references with content previews and metadata, response mode indicator

7.4 Vector Store and Document Processing

- **Purpose:** Ingest documents into a searchable vector representation and perform similarity-based retrieval
- **Description:** Extracts text from multiple file formats, splits into overlapping chunks, generates vector embeddings, stores in Qdrant with metadata, and retrieves by cosine similarity with per-conversation filtering
- **Key Functions:**
    - Process PDF, DOCX, TXT, and HTML files using specialized loaders
    - Split text into chunks (~1000 characters, 200-character overlap)
    - Generate 768-dimensional embeddings using Gemini Embedding 001 in batches of 5 with retry logic
    - Store chunks in Qdrant with metadata (source filename, conversation ID, chunk index)
    - Retrieve top-5 similar chunks filtered by conversation ID and active sources using Qdrant metadata filters
    - List all sources for a conversation with chunk counts
    - Delete all chunks associated with a specific source
- **Inputs:** Document files or raw text for ingestion; query strings with filter parameters for retrieval
- **Outputs:** Semantically similar document chunks for retrieval; operation status records for ingestion and deletion

7.5 Call Manager and Session Orchestration

- **Purpose:** Coordinate interactions between Teams bot, speech service, and RAG agent during active calls
- **Description:** Maintains state for each active call, implements multi-speaker detection and queuing, and logs all interactions as a structured transcript
- **Key Functions:**
    - Track active calls with status, participant count, event log, and transcript

    - Detect overlapping speakers when two speech events arrive within a configurable time window
    - Manage a FIFO speech processing queue — one question processed at a time
    - Forward dequeued questions to the RAG Agent and receive responses
    - Trigger TTS synthesis and audio playback after each response
    - Log every utterance with speaker name, timestamp, content, and bot/human flag
    - Provide call info and event history via API
- **Inputs:** Call event payloads from the Teams bot; meeting URLs for join operations
- **Outputs:** Call status records, transcript entries, orchestration commands to speech and RAG modules

7.6 Web Dashboard and User Interface

- **Purpose:** Provide a graphical interface for system administration, meeting management, document management, and interaction review
- **Description:** Single-page React application with two main views (Chat and Teams Meeting), dark theme with cyan-emerald gradient accents
- **Key Functions:**
    - **Chat Tab:**
        - Multi-conversation sidebar with pinned and recent groups
        - Create, rename, pin, and delete conversations
        - Source management panel with per-conversation document upload, toggle (checkboxes), and delete
        - Markdown-rendered bot responses with document/general knowledge badges
    - **Teams Meeting Tab:**
        - Meeting URL input with join/leave controls
        - ACS connection status indicator
        - Live transcript feed styled as a group chat with speaker avatars
        - Demo Q&A text input for testing without Azure setup
        - Event activity log with timestamps
    - **Shared Features:**
        - Text-based fallback when voice is unavailable
        - Real-time polling for bot status and call info

- **Inputs:** User interactions (clicks, text input, file uploads)
- **Outputs:** Rendered UI with real-time data from backend APIs

8. Tech Stack

Layer	Technology	Justification
Large Language Model	Google Gemini 2.5 Flash	Free tier with 1500 requests/day, low latency, high response quality
Embeddings	Gemini Embedding 001	Native compatibility with Gemini LLM, 768-dimensional vectors, same API ecosystem
RAG Framework	LangChain	Mature orchestration framework with abstractions for prompt construction, chain composition, and tool integration
Vector Database	Qdrant (Docker)	Purpose-built for vector similarity search, supports metadata-based filtering, lightweight resource footprint
Speech-to-Text	Vosk	Free, fully offline, no API key needed, eliminates network latency from recognition step
Text-to-Speech	gTTS	Free Google Translate TTS, no authentication required, natural-sounding English speech
Teams Integration	Azure Communication Services	Official Microsoft SDK for programmatic call joining, audio stream management, and event handling
Bot Identity	Azure AD App Registration	Required authentication mechanism for bot participation in Teams calls
Backend Framework	FastAPI (Python)	Asynchronous request handling, automatic API documentation, strong Pydantic validation
ASGI Server	Uvicorn	Optimized for high-concurrency Python applications
Frontend Framework	React 18 + Vite	Component-based architecture, fast hot module replacement, optimized production builds
UI Icons	Lucide React	Lightweight, consistent icon library
Markdown Rendering	React Markdown	Renders formatted bot responses with proper styling
Dev Tunneling	ngrok	Exposes local server for Azure webhook callbacks during development

9. Evaluation Strategy

The evaluation assesses four dimensions:

Response Accuracy:

- Upload a controlled set of organizational documents
- Submit a predefined test suite of questions with known correct answers
- Compare each response against the expected answer for factual correctness, completeness, and source attribution accuracy
- Distinguish between document-answerable and general-knowledge questions to verify correct response mode identification

Conversational Latency:

- Measure end-to-end time from completion of a participant's spoken question to the beginning of the agent's audible response

- This captures cumulative latency across STT, vector retrieval, LLM generation, and TTS synthesis
- Target: under 5 seconds for document-grounded responses, under 3 seconds for general-knowledge responses

Multi-Speaker Handling:

- Simulate call scenarios with two and three concurrent speakers
- Evaluate whether the system correctly detects overlapping speech, acknowledges all speakers, processes questions in order, and maintains coherence
- Monitor for failure modes: dropped questions, incorrect speaker attribution, confused context

Usability:

- Assess intuitiveness of document upload and management through the dashboard
- Evaluate clarity of transcript presentation and responsiveness of meeting controls
- Collect feedback from a small group of test users to identify areas for improvement

10. Applications

- **Corporate Onboarding** — Always-available knowledge assistant for orientation calls; answers policy and process questions; generates records of common queries for HR teams to improve documentation

- **Internal Training and Skill Development** — Attends technical training sessions; provides real-time answers from training materials; supplements trainers who may lack expertise across all topics

- **Customer Support and Service Operations** — Joins customer-facing calls; answers product questions from documentation and support articles; logs queries for analysis

- **Compliance and Regulatory Training** — Attends mandatory training sessions; provides answers grounded in regulatory documents; maintains auditable records of communicated information

- **Healthcare and Clinical Settings** — Participates in multidisciplinary team meetings; provides real-time references from clinical guidelines and protocols; maintains structured meeting minutes

- **Educational Institutions** — Participates in virtual classroom sessions; answers student questions from course materials; generates session transcripts for absent students

11. Conclusion

This project presents an AI-powered conversational agent designed to enhance productivity during Microsoft Teams meetings. By combining Retrieval-Augmented Generation (RAG) with real-time voice processing, the system enables users to query internal company documents hands-free during live meetings and receive accurate, context-aware responses.

Key outcomes of the project:

- Successful integration of Google Gemini 2.5 Flash for intelligent document-based Q&A
- A functional RAG pipeline using Qdrant vector database for fast, relevant retrieval
- Real-time speech-to-text and text-to-speech capabilities using Vosk and gTTS
- A clean, responsive web interface for both standalone chat and Teams meeting interaction

Future scope includes implementing per-session memory, multi-speaker queue management, meeting summarization, and deployment to a cloud environment for production use.

Signature of the Guide
Dr. V. Santhi